

MR
Multi-Edit Revisited
Userhandbuch v1.0

Projekt: <https://github.com/ebeneezer/MR>

22. Juni 2026

“Software and cathedrals are much the same – first we build them,
then we pray.”

Sam Redwine

Contents

1	Einfuehrung	5
1.1	Was ist MR?	5
1.2	Kernphilosophie	5
1.3	Hauptfeatures auf einen Blick	5
2	Installation	7
2.1	Systemvoraussetzungen	7
2.2	Installation aus dem Quellcode	7
2.3	Erster Start	7
3	Die MRMAC-Makrosprache	8
3.1	Einfuehrung in MRMAC	8
3.1.1	Der Makro-Compiler	8
3.1.2	Der Makro-Coprozessor	8
3.1.3	Beispiel: Einfaches MRMAC-Makro	8
3.2	Makro-Manager	8
4	Editieren wie ein Profi	9
4.1	Umgang mit grossen Dateien	9
4.2	Blocktypen	9
4.3	Code-Faltung und Smart-Indenting	9
4.4	Die Minimap	9
5	Fenster-Management	10
5.1	Der Fenster-Manager	10
5.2	Virtuelle Desktops	10
5.3	Inter-Window Copy & Paste	10
6	Workspaces und Projekte	11
6.1	Workspace-Management	11
6.2	Profile	11
6.3	Farbthemen	11

7	Suche und Ersetzung	12
7.1	Regulaere Ausdruecke	12
7.2	Rekursive Multifile-Suche	12
8	Compiler-Integration	13
8.1	Compiler-Profile	13
8.2	Fehler-Tracking	13
9	LSP-Integration	14
9.1	Language Server Protocol	14
9.2	LSP-Konfiguration	14
10	Tastatur und Keymapping	15
10.1	Key-Mapping-Manager	15
10.2	Emulationsmodi	15
11	Erweiterte Funktionen	16
11.1	Drucken und Export	16
11.2	Pipes und Shell-Kommandos	16
11.3	Automatische Sprachdetektion	16
12	Tastaturreferenz	17
13	Fehlerbehebung	18
13.1	Haeufige Probleme	18
14	Technische Architektur	19
14.1	Verwendete Technologien	19
14.2	Der Coprozessor im Detail	19
A	MRMAC-Sprachreferenz	20
A.1	Grundlegende Syntax	20
A.2	Editor-Kommandos	20
B	Lizenz und Danksagungen	21
B.1	Zitate	21
B.2	Projekt	21

List of Tables

2.1	Minimale Voraussetzungen	7
-----	------------------------------------	---

Chapter 1

Einfuehrung

1.1 Was ist MR?

MR ist eine Neuimplementierung des klassischen Programmier-Editors Multi-Edit fuer moderne Linux-Terminals. Der Editor verbindet klassische TUI-Bedienung mit heutigen Erwartungen an grosse Dateien, Syntaxanalyse, Language-Server-Integration, Projektprofile und reproduzierbare Einstellungen.

Der Schwerpunkt liegt nicht auf einer IDE-Oberflaeche, sondern auf einem schnellen, tastaturzentrierten Arbeitsfluss. Fenster, Makros, Suche, Compiler-Ausgaben und SideKicks bleiben im Terminal und sollen auch bei sehr grossen Dateien reaktionsfaehig bleiben.

1.2 Kernphilosophie

MR orientiert sich an drei praktischen Regeln:

- Wiederholbare Arbeit gehoert in Makros.
- Editorfunktionen muessen auch bei grossen Dateien berechenbar bleiben.
- Komfortfunktionen duerfen die direkte Bedienbarkeit nicht verdecken.

1.3 Hauptfeatures auf einen Blick

- eigene Makrosprache MRMAC mit Compiler und Coprozessor,
- Piecetable-basierte Bearbeitung grosser Dateien,
- Syntax-Highlighting, Folding, Outline und Smart-Indenting,
- Language-Server-Protocol fuer externe Sprachdienste,

- Compilerprofile mit Problems-Navigation,
- virtuelle Desktops, Workspaces und Farbthemen.

Chapter 2

Installation

2.1 Systemvoraussetzungen

Betriebssystem	Linux, 64 Bit empfohlen
Terminal	UTF-8-faehig mit ncursesw-Unterstuetzung
Compiler	C++20-faehiger Compiler, im Projekt bevorzugt <code>clang++</code>
Bibliotheken	ncursesw, PCRE2, TVision im Projektkontext

Table 2.1: Minimale Voraussetzungen

2.2 Installation aus dem Quellcode

```
git clone https://github.com/ebenezer/MR
cd MR
make clean all CXX=clang++
```

2.3 Erster Start

Nach dem Build kann MR direkt aus dem Arbeitsbaum gestartet werden. Die Einstellungen werden ueber die vorgesehenen Setup- und Runtime-Pfade geladen; Settings-Dateien sollen nicht durch externe Werkzeuge am Editor vorbei synthetisiert werden.

Chapter 3

Die MRMAC-Makrosprache

3.1 Einfuehrung in MRMAC

MRMAC ist die Makrosprache des Editors. Sie dient nicht nur der Tastenbelegung, sondern auch der Automatisierung wiederkehrender Editorablaufe. Makros werden kompiliert und danach durch die VM beziehungsweise den Coprozessor ausgefuehrt.

3.1.1 Der Makro-Compiler

Der Makro-Compiler uebersetzt Makroquellen in eine Form, die zur Laufzeit ausgefuehrt werden kann. Fehler sollen dabei frueh sichtbar werden, damit fehlerhafte Makros nicht erst in einer interaktiven Sitzung auffallen.

3.1.2 Der Makro-Coprozessor

Lang laufende oder deferred UI-nahe Operationen laufen nicht als lose Nebenlaeufigkeit durch die Anwendung. Der Coprozessor kapselt Ausfuehrung, Status und Rueckmeldung so, dass die VM-Grenzen nachvollziehbar bleiben.

3.1.3 Beispiel: Einfaches MRMAC-Makro

```
PROC HELLO_MR
  UI_MESSAGEBOX "MR" "Hello from MRMAC"
ENDPROC
```

3.2 Makro-Manager

Der Makro-Manager macht Makros auffindbar und ausfuehrbar. Fuer produktive Konfigurationen ist wichtig, dass die Runtime-Pfade und die persistierten Settings konsistent bleiben.

Chapter 4

Editieren wie ein Profi

4.1 Umgang mit grossen Dateien

MR ist auf sehr grosse Dateien ausgelegt. Die interne Textdarstellung vermeidet unnötige Vollkopien und erlaubt Navigation, Suche und Anzeige auch dann, wenn die Datei deutlich grösser als ein typischer Quelltext ist.

4.2 Blocktypen

Der Editor unterscheidet klassische Blockoperationen, Spaltenblöcke und zwischen Fenstern nutzbare Kopierpfade. Blockoperationen sollen vorhersehbar bleiben, auch wenn freie Cursorpositionen oder unterschiedliche Tabulatorbreiten im Spiel sind.

4.3 Code-Faltung und Smart-Indenting

Folding und Smart-Indenting werden sprachspezifisch berechnet. Für LaTeX sind insbesondere Umgebungen, Sections und Rohtextblöcke wie *verbatim* wichtig:

```
\section{Dies ist hier nur Text}
\begin{itemize}
  \item Auch dies bleibt Rohtext.
\end{itemize}
```

Der Editor darf diese innere Section nicht als echte Dokumentstruktur werten.

4.4 Die Minimap

Die Minimap ist eine kompakte Navigationshilfe. Sie ersetzt nicht die Outline, sondern ergänzt die schnelle Orientierung in langen Dateien.

Chapter 5

Fenster-Management

5.1 Der Fenster-Manager

Fenster sind ein zentraler Bestandteil des Arbeitsflusses. Editorfenster, Side-Kicks, Buildausgaben und Dialoge muessen ihre Rollen klar behalten.

5.2 Virtuelle Desktops

Virtuelle Desktops gruppieren Arbeitskontexte. Sie sollen schnelle Kontextwechsel erlauben, ohne Fensterzustand oder Projektbezug zu verlieren.

5.3 Inter-Window Copy & Paste

Kopieren zwischen Fenstern ist ein eigener Pfad und muss den Quell- und Zieltyp korrekt beachten. Das gilt besonders fuer Spaltenbloecke und freie Cursorpositionen.

Chapter 6

Workspaces und Projekte

6.1 Workspace-Management

Workspaces speichern Arbeitskontexte. Sie sind settingsnah, aber nicht automatisch Teil des kanonischen Kernsettings-Vertrags.

6.2 Profile

Profile bestimmen Sprache, Compiler, LSP und Editorverhalten fuer Dateitypen. Die Zuordnung muss nachvollziehbar sein, damit ein geoeffnetes Dokument nicht scheinbar zufaellig andere Editorregeln verwendet.

6.3 Farbthemen

Farbthemen betreffen die Darstellung, nicht die Semantik. Sie duerfen deshalb keine verdeckte Abhaengigkeit fuer Parser, Build oder Runtime-Logik bilden.

Chapter 7

Suche und Ersetzung

7.1 Regulaere Ausdruecke

MR verwendet PCRE2 fuer regulaere Ausdruecke. Suchen sollen schnell bleiben und Treffer so markieren, dass Navigation und Wiederholung stabil sind.

7.2 Rekursive Multifile-Suche

Die rekursive Suche ist fuer Projektarbeit gedacht. Trefferlisten muessen bedienbar bleiben und duerfen den Editor nicht blockieren.

Chapter 8

Compiler-Integration

8.1 Compiler-Profile

Compilerprofile beschreiben, wie ein Dokument oder Projekt gebaut wird. Fuer LaTeX ist `latexmk` der Standardpfad:

```
latexmk -pdf documentation/manuals/mr-userref.tex
```

8.2 Fehler-Tracking

Build-Ausgaben werden in Problems ueberfuehrt, wenn sie verwertbare Quellpositionen oder eindeutige Fehlermeldungen enthalten. LaTeX-Fehlerzeilen mit fuehrendem Ausrufezeichen und Warnungen mit `on input line N` sind fuer diesen Pfad wichtig.

Chapter 9

LSP-Integration

9.1 Language Server Protocol

Die LSP-Integration stellt Completion, Hover, Diagnostics und Snippets bereit. Fuer LaTeX ist `texlab` der bevorzugte Default-Kandidat, weil der Snippet-Pfad bei `\beg` die stabileren Ergebnisse liefert.

9.2 LSP-Konfiguration

Die LSP-Konfiguration wird ueber Profile und Settings wirksam. Middleware fuer Snippet-Formate gehoert zum jeweiligen Serverprofil, nicht in ad-hoc Dokumentlogik.

Chapter 10

Tastatur und Keymapping

10.1 Key-Mapping-Manager

Der Key-Mapping-Manager ordnet Tastenfolgen Editorfunktionen oder Makros zu. Resolver-Semantik und Persistenz sind geschuetzte Architekturteile.

10.2 Emulationsmodi

Emulationsmodi erleichtern Umstieg und Muskelgedaehtnis. Sie duerfen aber nicht dazu fuehren, dass interne Hotkeys hart codiert an UI-Texte gekoppelt werden.

Chapter 11

Erweiterte Funktionen

11.1 Drucken und Export

Druck- und Exportpfade sollen reproduzierbar sein. Fuer dieses Handbuch ist der LaTeX-PDF-Pfad zugleich ein Akzeptanzfall fuer Editor, Build und Diagnostics.

11.2 Pipes und Shell-Kommandos

Shell-Kommandos laufen ueber die konfigurierte Shell. Direkte Rueckfaelle auf `system()` oder `popen()` waeren fuer diesen Pfad falsch.

11.3 Automatische Sprachdetektion

Die Sprachdetektion verbindet Dateieindung, Profil und Syntaxregeln. Fuer `.tex`-Dateien muessen LaTeX-Regeln gelten, nicht zufaellig passende C- oder C++-Heuristiken.

Chapter 12

Tastaturreferenz

Alt-Enter	Snippet-SideKick-Auswahl uebernehmen
Tab	naechsten Snippet-Platzhalter ansteuern
Shift-Tab	vorherigen Snippet-Platzhalter ansteuern
F4	Suchtreffer beziehungsweise markierte Position fortsetzen

Chapter 13

Fehlerbehebung

13.1 Häufige Probleme

Wenn ein LaTeX-Build fehlschlägt, ist zuerst zwischen Paketfehler, Syntaxfehler, Warnung und Mehrfachlauf-Hinweis zu unterscheiden. Nicht jede LaTeX-Zeile ist ein navigierbares Problem, aber spezifische Fehlertexte müssen sichtbar bleiben.

Chapter 14

Technische Architektur

14.1 Verwendete Technologien

MR nutzt C++20 als Compilerstandard, bleibt stilistisch aber bewusst nahe an klassischem C++18. Externe Bibliotheken werden dort eingesetzt, wo sie einen klaren technischen Zweck erfüllen.

14.2 Der Coprozessor im Detail

Der Coprozessor trennt lang laufende Arbeit, UI-Rueckmeldung und VM-Grenzen. Diese Trennung ist fuer Buildausgaben, deferred UI und Makroausfuehrung entscheidend.

Appendix A

MRMAC-Sprachreferenz

A.1 Grundlegende Syntax

MRMAC-Programme bestehen aus Prozeduren, Anweisungen und Aufrufen von Editorfunktionen. Die genaue Semantik ist Teil der Makro- und VM-Vertraege.

A.2 Editor-Kommandos

Editor-Kommandos duerfen nicht als lose globale Registry neben den vorhandenen Dispatch- und Settingspfaden wachsen.

Appendix B

Lizenz und Danksagungen

B.1 Zitate

Dieses Handbuch enthaelt kurze historische Referenzen auf Multi-Edit und die Motivation hinter MR. Der Fokus der aktuellen Quelle liegt auf einem editierbaren LaTeX-Akzeptanzdokument.

B.2 Projekt

Projektseite: <https://github.com/ebenezer/MR>